

Programming the Pico Computer 3 in C

Fuzix on the RP2350B

What this is

Everything a C programmer needs to reach the facilities this port adds to Fuzix: the graphics modes, the off-screen framebuffer, text at pixel positions, sound, the joystick and ADC, and the 8 MiB of PSRAM.

None of it is a library call. The kernel owns the hardware and exposes it through **ioctl**s on **/dev/sys**, so everything below is the same three lines:

```
#include <fcntl.h>
#include <sys/ioctl.h>
#include "pico_ioctl.h"           /* the numbers and structures */

int sys = open("/dev/sys", O_RDWR);
ioctl(sys, GFXIOC_MODE, &mode);
```

pico_ioctl.h is the authority. If this document and that header disagree, the header is right.

Two ways to write C here

There are two, and they are genuinely different environments.

On the board: cc

```
$ cc -o prog.bc prog.c
$ ./prog.bc
```

The Fuzix C compiler runs on the machine itself and produces bytecode with native ARM Thumb spans, executed by **bcrun**. This is what **mmbc** targets and what most PC3 programs are.

What you get:

- headers: **stdio.h**, **stdlib.h**, **string.h**, **assert.h**, **limits.h**, **stddef.h**, **math.h**, **ctype.h**, **stdint.h**, **time.h** — in **/usr/lib/cc/include**, describing what **bcrun** provides rather than what Fuzix's own **libc** does.

- a C library subset resolved **by name at load time**: `printf`, `sprintf`, `fprintf`, `fopen`/`fclose`/`fread`/`fwrite`/`fgetc`/`fgets`/`fputc`/`fputs`/`fseek`/`ftell`/`feof`, `malloc`/`calloc`/`realloc`/`free`, the `str*` and `mem*` family, `atoi`/`atof`/`atol`/`strtol`/`strtod`, `open`/`close`/`read`/`write`/`lseek`/`creat`/`remove`/`rename`/`unlink`, `exit`, `rand`/`srand`, `time`.
- two of our own: `adval(n)` and `time_us()` / `time_us64()`.

Because names resolve at load time, **declaring a function is all the header you need**:

```
extern long time_us(void);      /* no header required */
```

The catch: **ioctl is not in that table**. A `bcrun` program cannot call it, so it cannot reach the graphics or PSRAM ioctls directly. What it gets instead is `adval()` and `time_us()`, which `bcrun` implements on its behalf. To drive the display from a `bcrun` program, write `MMBasic` and let `mmbc` translate it — the runtime does the ioctls.

Cross-compiled: native ARM binaries

Everything in `Kernel/platform/platform-rpipico/utils/` is built this way, and this is the path for a program that needs the full syscall interface:

```
arm-none-eabi-gcc -mcpu=cortex-m33 -Os -fno-builtin \
    -isystem $FUZIX/Library/include -c prog.c
arm-none-eabi-ld $FUZIX/Library/libs/crt0_armm0.o prog.o -o prog \
    -L$FUZIX/Library/libs -lcarmm0 -pie -static -no-dynamic-linker \
    -z max-page-size=4 -T $FUZIX/Library/elfexe32.ld
```

or just add it to `utils/Makefile` and `make`. These get Fuzix's own `libc`, `ioctl`, and every syscall. **Everything in the rest of this document assumes this path.**

Ask for more stack with `-z stack-size=N` if you recurse; the default is 8 KB and the ELF loader honours the request up to 64 KB.

Worked examples on disc

Read these before writing anything; they are all short.

program	shows
<code>utils/gfxttest.c</code>	modes, palette, <code>GFXIOC_BLIT</code>
<code>utils/linebench.c</code>	<code>GFXIOC_PIXEL</code> against <code>GFXIOC_PIXELS</code> , with timings
<code>utils/ripple.c</code>	per-pixel drawing versus a shadow buffer
<code>utils/bmtest.c</code>	<code>GFXIOC_BITMAP</code> and <code>GFXIOC_RECT</code> , verified by readback

program	shows
utils/saveimage.c, loadimage.c	reading and writing the screen
utils/memprobe.c	how much memory a process can actually have
utils/allocbench.c	the cost of a PSRAM arena allocation

The memory a program gets

- The process pool is **340 KB**, and one process may use nearly all of it — `memprobe` reports about **292 KB**. The ceiling is not a constant: a grow must leave room for the largest *other* process to be resident (`largest_neighbour()` in `swapper.c`), so it moves with what else is running.
- Up to **64 processes**.
- Memory is packed in 4 KB chunks at actual size. A swapped-out process gets a PSRAM allocation of exactly its own size.
- `sbrk()` grows the break and fails when there is no room. Probing with `sbrk(4096)` in a loop until it refuses is a legitimate way to find your ceiling — `bbcbasic` does exactly that.
- **There is no MMU**. A wild pointer corrupts the kernel and takes the machine down with no diagnostic. This is the single most important thing to know.

Graphics

Modes

```
int mode = 7;
ioctl(sys, GFXIOC_MODE, &mode);      /* pass a POINTER to an int */
```

mode	resolution	colours	raster
0, 3	640×256, 1bpp	2	1024×768
1, 4	320×256, 4bpp	16	1024×768
2, 5	160×256, 4bpp	16	1024×768
7	320×240, 4bpp	16	640×480
0xFF	back to the 80×40 text console		640×480

Modes 0–5 are the BBC Micro set. **Mode 7 is not teletext** — it is 320×240 in 16 colours, and it is what MMBasic calls `MODE 2`.

Switching *within* a raster keeps the monitor locked; crossing between 640×480 and 1024×768 restarts the scanout and the monitor resyncs.

Geometry, so nothing has to hardcode it:

```

struct gfx_info gi;
ioctl(sys, GFXIOC_INFO, &gi);
/* gi.width, gi.height, gi.stride, gi.bpp, gi.mode */

```

Colour

Everything speaks **RGB888** and the kernel converts to the mode's own colours — you never deal in colour indices.

```

ioctl(sys, GFXIOC_COLOUR, (void *)0xFF8000L); /* the VALUE, not a pointer */

```

Mode 7's sixteen colours are MMBasic's RGB121 set: red and blue are 0 or 255, green has four levels. Anything else maps to the nearest, which is worth knowing before you ask for RGB(r, g, 99) and get black — a blue of 99 is nearer 0 than 255 every time.

Drawing

```

ioctl(sys, GFXIOC_PIXEL, (void *)GFX_PIXEL_PACK(x, y)); /* current colour */
int c = ioctl(sys, GFXIOC_GETPIXEL, (void *)GFX_PIXEL_PACK(x, y));

struct gfx_rect r = { x1, y1, x2, y2 };
ioctl(sys, GFXIOC_RECT, &r); /* filled, current colour */

```

GFX_PIXEL_PACK puts the coordinates in the ioctl *argument*, so there is nothing to copy in and nothing to validate: measured at 1.30 μ s against 1.488 μ s for an ordinary ioctl. Its y is 9 bits, so it stops at 511 — use the batch calls for the 768-line modes.

A syscall costs 1.3 μ s and a pixel store costs 15 ns. Anything with more than a handful of points should be batched:

```

struct gfx_pt pts[512];
struct gfx_batch b = { .count = n, .flags = 0,
                      .items = pts, .colours = NULL };
ioctl(sys, GFXIOC_PIXELS, &b); /* a run of points */
ioctl(sys, GFXIOC_RECTS, &b); /* a run of rectangles: gfx_rc[] */

```

colours may be NULL for “all in the current colour”, or one RGB888 per item. Up to GFX_BATCH_MAX (1024) items. A 312-point line costs 433 μ s one at a time and 71 μ s batched.

The arrays are read where they lie — nothing is copied into the kernel.

Bitmaps and text

```

struct gfx_bitmap gb = { x, y, w, h, scale, 0, fg, bg, bits };
ioctl(sys, GFXIOC_BITMAP, &gb);

```

Bits are MSB-first, row-major — **exactly MMBasic’s font packing**, so fonts interchange. `bg = -1` is transparent. Source up to `GFX_BITMAP_MAX`.

For text there is no need to carry a font:

```
struct gfx_text gt = { x, y, scale, 0, fg, bg, len, str };
int endx = ioctl(sys, GFXIOC_TEXT, &gt);
```

This draws a run of characters in the console’s own font (MMBasic’s `font1`, 8×12) and returns the `x` it ended at. `bg = -1` leaves the paper alone. Up to `GFX_TEXT_MAX` (256) characters, one crossing.

```
ioctl(sys, GFXIOC_SCROLL, (void *)((rows << 24) | rgb888));
```

Scrolls the write target: `rows` signed in the top byte, positive up. This is the same call the console uses for its own scrolling, so a program and the shell move the picture the same way.

The off-screen framebuffer

Draw off-screen, show it in one go. This is MMBasic’s `FRAMEBUFFER`.

```
ioctl(sys, GFXIOC_FBOPEN, (void *)1);    /* claim it */
ioctl(sys, GFXIOC_FBSEL, (void *)1);     /* draw off-screen */
... every drawing call above now goes to the buffer ...
ioctl(sys, GFXIOC_VSYNC, (void *)0);     /* optional: top of frame */
ioctl(sys, GFXIOC_FBCOPY, (void *)0);     /* buffer -> screen */
ioctl(sys, GFXIOC_FBSEL, (void *)0);     /* back to the screen */
ioctl(sys, GFXIOC_FBOPEN, (void *)0);    /* give it back */
```

Things to know:

- **The layer is owned.** `FBOPEN` fails with `EBUSY` if another process holds it, `EINVAL` on a board with no PSRAM. It is released automatically on exit, on exec, and on a mode change — so a program that dies does not leave the machine drawing into nowhere.
- **The write target is per process.** Anything else that draws while you are between frames — another program, the shell — still goes to the screen. You cannot have your picture scribbled on.
- **CREATE belongs after MODE.** A mode change discards the buffer, because its contents are in the geometry of the mode being left.
- `FBCOPY` takes 1 to copy screen→buffer instead.

What it costs, measured on the board in mode 7:

full-screen fill, to the screen (SRAM)	1.49 ms
the same into the buffer (PSRAM)	3.63 ms
copy the 38,400-byte buffer to the screen	0.73 ms (53 MB/s)

So a redraw-and-show frame is about 4.4 ms against the 17 ms the display gives you. `GFXIOC_VSYNC` before the copy holds a loop to the refresh rate — 59 fps with room for roughly three times as much drawing.

Sound

Four channels, BBC SOUND semantics.

```
struct snd_cmd s = { .chan = 1, .amp = -15, .pitch = 100, .dur = 20 };
ioctl(sys, SNDIOC_SOUND, &s);      /* -1/EAGAIN if that queue is full */
ioctl(sys, SNDIOC_ENV, envbytes);   /* 14 bytes: N,T,PI1..3,PN1..3,AA,AD,AS,AR,ALA,ALD */
ioctl(sys, SNDIOC_QUIET, 0);        /* silence everything */
```

`amp` is -15..0 for volume, or 1..16 to use `ENVELOPE` `n`. The channel word carries the `&1x` flush and `&Sxx` sync bits.

Joystick, ADC and the clock

```
int v = ioctl(sys, PICOIOC_ADVAL, (void *)n);
```

n	reads
0	joystick switches GP34–37, pressed = 1, bit 0 = GP34
1–4	ADC on GP41–44, 16-bit (12-bit shifted left 4)
-5..-8	free slots in sound queue 0–3
-9	microsecond counter, 31 bits in the return value
-10	the full 64-bit counter: pass an 8-byte buffer whose low word holds -10

PSRAM

8 MiB, outside the process image, reached as a raw address.

```
struct psram_req rq = { .len = 64 * 1024 };
ioctl(sys, PSRAMIOC_ALLOC, &rq);      /* rq.base is the address */
ioctl(sys, PSRAMIOC_REALLOC, &rq);    /* rq.base in, the NEW base out */
ioctl(sys, PSRAMIOC_FREE, (void *)rq.base);
```

```
struct psram_stat st;
ioctl(sys, PSRAMIOC_STAT, &st);       /* total, free, largest */
```

- Allocation is 4 KB granular and **released on exec and exit**; a fork leaves the arena with the parent.

- **REALLOC may move the block** — newlib copies when it cannot extend in place — so rebuild any interior pointers.
- The address is raw and unprotected; there is no MMU.
- An alloc+free pair costs about **5.1 μ s**, against 350 ns for an in-process malloc. Ask once and sub-allocate, rather than per call.
- **PSRAM is roughly 2.4 $\times$ slower than SRAM** for scattered access, but a bulk sequential copy runs at 53 MB/s. Put big, streamed things there; keep hot, randomly-accessed things in your own memory.

Miscellaneous

```
ioctl(sys, PICOIOC_FLASH, 0);           /* reboot into BOOTSEL */
ioctl(sys, PICOIOC_KBDMAP, "uk");       /* US UK DE FR ES BE */
```

Traps worth knowing

No MMU. Said once already, worth saying twice. A bad pointer in a user program takes the whole machine down with no message.

A panic blanks the HDMI screen. plt_monitor() resets core1 and with it the scanout, so a panic looks like a dead display. **The message only ever goes out of the serial port.** If a program kills the machine, read the UART.

Text and graphics share the framebuffer. In a graphics mode the console draws into the screen buffer, so printf from your program lands on the picture and scrolls it when the line wraps. Use GFXIOC_TEXT for text you want positioned, and keep printf for the serial console.

Timing is contended. core1 DMAs scanlines out of SRAM continuously, so RAM bandwidth is shared. This cuts both ways: it is why the kernel now executes from flash at no measurable cost, and why a benchmark that touches the framebuffer will not be reproducible to the last percent.

The compiler is small. cc on the board has real limits — deeply nested expressions and very large functions can exceed it. It also has a threshold beyond which a function stops being compiled to native ARM and falls back to bytecode, which is worth 2–3 $\times$ on a hot loop. If a small change makes a program suddenly slower, that is the first thing to suspect; BCRUN_BYTECODE=1 forces bytecode so you can compare.

Where things live

Kernel/platform/platform-rpipico/	
pico_ioctl.h	every ioctl, and the authority
display.c/.h	modes, primitives, the framebuffer
console.c	the terminal, and the font

swapper.c	the process pool
psram.c, arena.c	PSRAM and the arena allocator
utils/	the worked examples above
PC3-GFX-DESIGN.md	why the graphics interface is shaped as it is
PC3-FLASH-PLAN.md	what runs from flash and what may not
BUILDING-PC3.md	building the kernel and the card